

GoldVllm – Arbeitsanweisung für KI-Assistenten

GoldVllm_Anleitung_KI.md

Stand: 25.09.2026 · Zielgruppe: KI-Agenten mit Shell-Zugriff auf ein NVIDIA-GB10-System (DGX Spark, ASUS Ascent GX10 o. ä.) Englische Kurzfassung derselben Regeln: `AI_AGENT_GUIDE.md` im Repo-Wurzelordner. Bei Widerspruch gilt diese Datei für die Befehle, `AI_AGENT_GUIDE.md` für die Regeln – beide sind inhaltlich gleich gemeint.

0 Auftrag und Vorrang

Du sollst GoldVllm installieren, betreiben, prüfen oder tunen. GoldVllm ist ein **eingefrorener, gemessener Referenzstand**. Dein Standardziel am Ende jeder Sitzung: **der Referenzstand läuft und `gates.py` meldet `GATES OK`**, ausser der Nutzer hat ausdrücklich anders entschieden.

Vorrang bei Konflikten: 1. Die harten Regeln in Abschnitt 2. 2. Ausdrückliche Anweisungen des Nutzers – aber **vorher** sagen, wenn eine Anweisung eine harte Regel bricht, und die Folge nennen. 3. Diese Anleitung. 4. Deine eigenen Annahmen. Annahmen sind zu prüfen, nicht auszuführen.

1 Referenzstand (Soll-Werte)

Schlüssel	Soll-Wert
Image	<code>gx10-vllm:goldvllm</code> , gebaut aus <code>blazux/qwen3.8-Flash-DGX@d542745</code> , <code>Dockerfile.v0.29</code> , Basis <code>vllm/vllm-openai:v0.29.0</code>
vLLM-Version im Image	<code>0.29.0</code>
Modell	<code>RadixArk/Qwen3.8-Flash-Next-NVFP4</code> @ <code>7b719225242aacd3dbd3f9407468c2ee9a9d2594</code> (419 Dateien, 135.3 GB)
Modell-Manifest	<code>data/model-snapshot-7b719225.tsv</code> (Datei, Blob, Bytes; Safetensors-Blob = SHA256)
Draft-Vokabular	<code>draft_vocab_de_65536.npy</code> , 262'272 Bytes, SHA256 <code>a864739485e0804049fe3481eff53770157f4b76cd6d91f556ff91ce454a23a1</code>
Container-Name	<code>qwen38-flash</code> , <code>--restart no</code> , Port nur <code>127.0.0.1:8000</code>
Startzeile	ausschliesslich <code>config/run-goldvllm.sh</code> (35 vLLM-Argumente + Umgebungsvariablen, identisch mit Produktion)
KV-Cache	<code>--kv-cache-memory-bytes 12884901888</code> (12 GiB, 424'354 Tokens)
MTP	<code>{"method": "mtp", "num_speculative_tokens": 2}</code>
Kontext	<code>--max-model-len 262144</code>
RAM-Wächter	Schwelle 122 GB belegt (dezimal), Intervall 10 s
Kaltstart bis bereit	852–902 s
Plattform (getestet)	Kernel <code>7.0.0-1019-nvidia</code> , Treiber 580.178.04, CUDA 13.0, Docker 29.1.3, nvidia-container-toolkit 1.20.0

Referenz-Messwerte (Mittel GOLD1/GOLD2, `data/goldvllm-baseline-2026-09-25/`):

Metrik	Wert	Toleranz für „unverändert“
G30	61.7 s	±13 % Einzellauf
Agent 30k / 80k	28.4 / 48.4 s	±13 % Einzellauf
TTUA Median / p95	1.47 / 1.78 s	–
TTFT kalt 30k / 80k / 110k	13.0 / 35.6 / 52.3 s	–
TTFT warm 30k / 80k	1.0 / 1.7 s	–
Decode	27.4 tok/s	±5 % Einzellauf
TP1 / TP2 / TP4	23 / 32 / 52 tok/s	–
Qualität	12/12	exakt , 11/12 ist ein Fehler
MemAvailable min unter Last	~15.5 GiB	Warnung unter 10 GiB
Swap max	~7 GB	–
Xid / OOM / Engine-Fehler	0 / 0 / 0	exakt

2 Harte Regeln

#	Regel	Prüfung / Umsetzung
R1	Nie zwei Inferenz-Engines gleichzeitig. GPU und System teilen 128 GB	vor jedem Start: <code>nvidia-smi --query-compute-apps=pid --format=csv,noheader</code> muss leer sein
R2	Vor jeder Änderung sichern.	jede berührte Datei kopieren, <code>docker inspect qwen38-flash</code> speichern, Image-ID und Modell-Revision notieren, Rückweg aufschreiben – bevor du änderst
R3	<code>/health 200</code> ist kein Korrektheitsnachweis.	vor jeder Messung und nach jeder Änderung <code>bench/gates.py</code> → <code>GATES OK</code> , plus ein echter Tool-Call über den tatsächlichen Client
R4	Eine Variable pro Test.	Startargumente von A und B diffen; genau eine Zeile darf abweichen
R5	Nur paarweise Vergleiche.	ABAB (A1 B1 A2 B2), frische Container, <code>drop_caches</code> , Aufwärmen. Entscheidungsregel vor der Messung schriftlich festlegen
R6	RAM-Wächter bleibt an, Schwelle bleibt 122 GB.	Speicher über festen KV-Cache begrenzen, nicht über den Wächter. Bei 126 GB kam <code>CUBLAS_STATUS_INTERNAL_ERROR</code> statt sauberem Stopp
R7	systemd besitzt den Produktionscontainer.	Testcontainer heißen <code>bench-*</code> , anderer Port. Produktion wird nur auf ausdrückliche Nutzerentscheidung ersetzt
R8	Produktion hängt nicht an einem Git-Arbeitsordner.	Betriebsskripte nach <code>/usr/local/sbin</code> bzw. <code>/opt/<name></code> installieren
R9	Wahrheitsgetreu berichten.	„gemessen“, „abgeleitet“, „laut Upstream“ sauber trennen. Übersprungene oder gescheiterte Schritte nennen
R10	Rauschen ist kein Gewinn.	unter der Rauschgrenze lautet das Ergebnis „kein messbarer Effekt“
R11	Geheimnisse bleiben lokal.	API-Key nur in <code>~/config/vllm.key</code> (Modus 600), nie in Logs, Commits, Berichte oder Befehlszeilen-Ausgaben kopieren
R12	Kein Kernel-Update ohne Rückfrage.	GX10 hat kein DKMS für die NVIDIA-Module; nach einem Kernel-Update fehlen GPU und Desktop

3 Installation – Ablauf mit Abbruchbedingungen

Führe die Schritte der Reihe nach aus. Gehe erst weiter, wenn die Spalte „Erwartet“ erfüllt ist. Bei „Abbruch“ stoppen und dem Nutzer berichten.

Schritt	Befehl	Erwartet	Abbruch wenn
I1 Plattform	<code>uname -r; nvidia-smi --query-gpu=name,driver_version --format=csv,noheader; docker --version</code>	GB10, Treiber 580.x, Docker vorhanden	keine GPU. Abweichende Versionen: melden, nicht abbrechen
I2 GPU im Container	<code>docker run --rm --gpus all --entrypoint nvidia-smi vllm/vllm-openai:v0.29.0</code>	GB10 sichtbar	Fehler → nvidia-container-toolkit prüfen
I3 Platz	<code>df -h ~/.cache/huggingface/var/lib/docker</code>	≥ 200 GB frei	< 200 GB
I4 Swap	<code>swapon --show</code>	~16 GB	kein Swap: melden
I5 Image	<code>git clone https://github.com/blazux/qwen3.8-Flash-DGX.git && cd qwen3.8-Flash-DGX && git checkout d542745 && DOCKER_BUILDKIT=1 docker build -f Dockerfile.v0.29 -t gx10-vllm:goldvllm .</code>	Build erfolgreich	ohne BuildKit nicht bauen (Legacy-Builder: ~30 GB Lesen je Schritt, ≈ 2 h). BuildKit-Dauer nicht gemessen. Das Referenz-Image entstand stattdessen per <code>build/baue_b_inner.sh</code> (wörtliche Übertragung desselben Dockerfiles, 100 s; Gleichheit der Image-Inhalte nicht Datei für Datei geprüft)
I6 Version	<code>docker run --rm --entrypoint python3 gx10-vllm:goldvllm -c "import vllm; print(vllm.__version__)"</code>	0.29.0	andere Version
I7 Modell	<code>hf download RadixArk/Qwen3.8-Flash-Next-NVFP4 --revision 7b719225242aacd3dbd3f9407468c2ee9a9d2594</code>	419 Dateien, 135.3 GB	Dateiliste/Größen weichen von <code>data/model-snapshot-7b719225.tsv</code> ab
I8 Vokabular	<code>sudo install -D -m 0444 draftvocab/draft_vocab_de_65536.npy /opt/gx10/draftvocab/draft_vocab_de_65536.npy && sha256sum /opt/gx10/draftvocab/draft_vocab_de_65536.npy</code>	SHA256 a8647394...54a23a1	SHA falsch
I9 Key	<code>python3 -c 'import secrets; print(secrets.token_hex(32))' > ~/.config/vllm.key && chmod 600 ~/.config/vllm.key</code>	Datei vorhanden, Modus 600	–
I10 Speicher frei	<code>grep MemAvailable /proc/meminfo</code>	≥ 90 GB	darunter: Verursacher suchen, nicht starten
I11 Start	<code>VOCAB=/opt/gx10/draftvocab/draft_vocab_de_65536.npy KEYFILE=~/.config/vllm.key ./config/run-goldvllm.sh</code>	gestartet: qwen38-flash	Exit 2 Key fehlt · 3 Modell fehlt · 4 Vokabular/SHA · 5 GPU belegt

Schritt	Befehl	Erwartet	Abbruch wenn
I12 Warten	<code>docker logs -f qwen38-flash</code>	<code>Application startup complete</code> nach ~15 min	Container beendet sich → letzte 40 Logzeilen lesen, nicht blind neu starten. Laden nicht unterbrechen
I13 Endpunkte	<code>curl -s -o /dev/null -w '%{http_code}' localhost:8000/health</code> und <code>/v1/models</code> ohne und mit Key	200 / 401 / 200	anderes Ergebnis
I14 Gates	<code>cd bench && VLLM_API_KEY=\$(cat ~/.config/vllm.key) KORPUS_DIR=<txt-ordner> python3 gates.py --url http://127.0.0.1:8000 --label install --ausgabe gates.json</code>	GATES OK	alles andere
I15 Wächter	<code>llm-memory-guard.sh</code> → <code>/usr/local/sbin/</code> , <code>.service</code> / <code>.timer</code> → <code>/etc/systemd/system/</code> , <code>systemctl enable --now llm-memory-guard.timer</code>	Timer aktiv	–
I16 Autostart	feste Kopie des Repos nach <code>/opt/goldvllm</code> ; <code>config/systemd/llm-server.service</code> → <code>/etc/systemd/system/</code> , <code>DEIN_NUTZER</code> ersetzen; <code>systemctl enable llm-server</code>	Unit enabled	Wächter nicht aktiv → Autostart nicht einschalten
I17 Recovery	<code>llm-recovery.sh</code> → <code>/usr/local/sbin/</code> , <code>.service</code> / <code>.timer</code> → <code>/etc/systemd/system/</code> , <code>llm-recovery.default</code> → <code>/etc/default/llm-recovery</code> , dort <code>LLMR_KEYFILE</code> setzen; optional <code>LLMR_HEALTH_CMD</code> (z. B. <code>gates.py</code>) + <code>LLMR_HEALTH_USER</code> ; <code>systemctl enable llm-recovery.timer</code>	<code>llm-recovery.sh --status</code> zeigt <code>NORMAL</code> , Journal ohne „LLMR_KEYFILE nicht gesetzt“	–
I18 Reboot-Test	vorher manuell gestarteten Container stoppen (<code>docker stop qwen38-flash</code> , sonst Exit 5 beim Unit-Start), nach Rückfrage beim Nutzer neu starten, ~15 min warten, I13 + I14 wiederholen	wieder <code>GATES OK</code> ohne Eingriff	kommt nicht hoch → Journal von <code>llm-server</code> lesen
I19 Baseline	Messung nach Abschnitt 6	Werte im Toleranzbereich von Abschnitt 1	stark abweichend → melden, nicht „tunen“

Recovery-Gesundheitsprüfung: Ohne `LLMR_HEALTH_CMD` prüft die Recovery nur `/health` = 200 und `/v1/models` mit Key = 200 – das ist nach R3 **kein** Korrektheitsnachweis. Empfehlung an den Nutzer: `gates.py` als `LLMR_HEALTH_CMD` eintragen. Die Recovery setzt `XDG_RUNTIME_DIR=/run/user/<uid>` für `LLMR_HEALTH_USER` selbst (sonst scheitert dort `systemctl --user` mit „Failed to connect to bus“). Testmodus ohne Wirkung: `LLMR_SIMULATE=1 LLMR_STATE_DIR=<tmp>` plus `SIM_*`-Variablen (Liste im Skriptkopf).

4 Betrieb – Zustandsmaschine

```
+-----+
Boot -----> | llm-server start | -- ExecStartPre wartet auf nvidia-smi (60 s)
+-----+
| run-goldvllm.sh, bis 40 min auf /v1/models = 401
v
+-----+      belegter RAM >= 122 GB
|   NORMAL   | -----+
+-----+      |
^              v
|              +-----+
|              | STOPPED (Wächter hat |
|              | alle LLM-Server beendet)|
|              +-----+
|              | Recovery-Timer, jede Minute
|              v
|              3x MemAvailable >= 114'000 MiB, GPU ok, kein neuer Xid,
|              >= 10 min Abstand, <= 2 Versuche / 6 h
+-----+ RECOVERING <---- WAIT_MEM
|
| Budget aufgebraucht
v
FAILED -- bleibt bis `llm-recovery.sh --reset`
```

Stilllegen der Recovery ohne Deinstallation: `touch /etc/llm-recovery.disabled`.

5 Fehlerbehandlung – Entscheidungstabelle

Symptom	Erste Aktion	Nicht tun
Container startet nicht / beendet sich	<code>docker logs --tail 40 qwen38-flash</code> ; R1 prüfen; <code>MemAvailable ≥ 90 GB</code> ?	blind neu starten, Argumente „ausprobieren“
<code>run-goldvllm.sh</code> Exit 5	fremden GPU-Prozess identifizieren, Nutzer fragen, ob er beendet werden darf	fremde Prozesse ungefragt beenden
Wächter hat gestoppt	<code>journalctl -t llm-memory-guard</code> , Verursacher suchen (zweite Engine, Test, grosser Prozess)	Schwelle anheben, Wächter abschalten
Ausgabe auffällig (Wiederholung wie <code>!!!!</code> , leerer Text, falsche Zahlen)	Messungen stoppen, <code>gates.py</code> laufen lassen, bei Fehler Referenzstand wiederherstellen	weiter benchmarken, Ergebnis trotzdem melden
Gates schlagen fehl	berichten mit exakter Ausgabe; Rückweg aus R2 anwenden	Gate lockern oder Aufgabe überspringen
Nach Reboot keine GPU	<code>uname -r</code> gegen <code>7.0.0-1019-nvidia</code> ; NVIDIA-Module geladen? (<code>lsmod</code> muss <code>nvidia</code> -Einträge zeigen)	Treiber neu installieren ohne Rückfrage
<code>NV_ERR_NO_MEMORY</code> im Kernel-Log beim Laden	ignorieren, sofern kein Xid folgt	als Fehler melden
Recovery greift nicht	<code>llm-recovery.sh --status</code> , <code>journalctl -t llm-recovery</code> ; <code>LLMR_KEYFILE</code> in <code>/etc/default/llm-recovery</code> gesetzt und lesbar?	Recovery-Budget hochsetzen

Symptom	Erste Aktion	Nicht tun
Recovery hält gesundes vLLM für krank	<code>LLMR_HEALTH_CMD</code> von Hand als <code>LLMR_HEALTH_USER</code> ausführen, Exit-Code und Ausgabe lesen	Prüfung entfernen, um die Recovery „grün“ zu bekommen
KV-Wert nach Reboot anders	Startargumente per <code>docker inspect</code> gegen <code>run-goldvllm.sh</code> diffen; Werte an einer Stelle pflegen	zweite Konfigurationsquelle anlegen

6 Messen und Tunen

6.1 Messkette

```
cd bench
export VLLM_BASIS_URL=http://127.0.0.1:8000 VLLM_API_KEY=$(cat ~/.config/vllm.key) VLLM_CONTAINER=qwen38-flash \
    KORPUS_DIR=<txt-ordner> FUELLTEXT=<datei.txt> AUFWAERMTEXT=<datei.txt>
python3 gates.py --url $VLLM_BASIS_URL --label <lauf> --ausgabe gates.json # muss GATES OK sein
python3 aufwaermen.py --container qwen38-flash --hoechstdauer 900
python3 benchmark_gx10.py --nur-schnell --agent-schritte 30 --ausgabe g30.json # G30, TTUA, TP1/2/4
python3 nacht_bench.py --url $VLLM_BASIS_URL --label <lauf> --ausgabe nacht.json # Decode, TTFT, Agent
30k/80k, Qualität
```

- `FUELLTEXT` / `AUFWAERMTEXT` **setzen**. Ohne sie nutzt das Kit einen Platzhaltersatz, der mit Prefix-Caching zu gute Zahlen liefert.
- Die Langkontext-Dokumente werden einmal kalibriert und in `bench/dokumente_cache.json` abgelegt. Die Kalibrierung braucht vLLMs `/tokenize` – deshalb zuerst gegen vLLM laufen lassen, dann bekommen alle Läufe byte-gleiche Prompts.
- `sampler.py` und `psi_sampler.py` parallel mitlaufen lassen. `sampler.py --stoppe <container>` stoppt einen Testcontainer bei Speicher Gefahr.
- Auswertung: `auswertung.py` (Kennzahlen je Lauf), `entscheide.py` (E2E-Score, ABAB-Entscheid).

6.2 Entscheidungsgrösse

E2E = geometrisches Mittel der Zeitverhältnisse Kandidat/Referenz für G30, Agent 30k und Agent 80k. Gewinn in % = $(1 - E2E) \times 100$. Decode und TTFT werden berichtet, entscheiden aber nicht allein. Qualität 12/12 und Gates OK sind Pflicht für jeden Lauf.

6.3 Ablauf eines Vergleichs

1. Entscheidungsregel schriftlich festlegen (z. B. „B gewinnt, wenn beide Paare E2E-Gewinn > 3 % und Qualität 4×12/12“).
2. R2 sichern. Produktion stoppen (R1). Testcontainer `bench-<name>` auf anderem Port.
3. Pro Lauf: frischer Container → `sync; echo 3 > /proc/sys/vm/drop_caches` → Aufwärmen → Gates → Messkette.
4. Reihenfolge A1 B1 A2 B2.
5. Entscheiden nach der vorher festgelegten Regel. Widersprechen sich die Paare (ein Paar besser, eines schlechter): „kein messbarer Effekt“.
6. Referenzstand wiederherstellen, Gates OK, Bericht.

6.4 Bereits entschieden – nicht erneut testen ohne neuen Grund

Hebel	Ergebnis	Status
offizielles vLLM 0.29.0 statt Qwen-Preview-Build	TTFT 30k–116k –32 bis –40 %, Agent 80k –26 %	übernommen
KV fest 12 GiB statt 18.74 GiB	≥ 9.3 GiB Abstand zum Wächter statt ~570 MiB	übernommen
MTP 1 / 3 / 4	Agentenschritte +18 % / Decode –12 % / –16 %	abgelehnt, MTP 2 bleibt

Hebel	Ergebnis	Status
deutsches Draft-Vokabular	E2E +3.95 % / +6.28 %, Mittel +5.11 % (ABAB)	übernommen
<code>VLLM_PLE_MMAP_FAST_ROWS=0</code>	schlechter	abgelehnt
deterministischer Top-k-Kernel (<code>VLLM_QSA_DET_TOPK</code>)	G30 +5 %, TTUA p95 +19 %, kein gemessener Nutzen	abgelehnt
<code>VLLM_QSA_EXACT_TOPK=1</code>	laut blazux –20 bis –40 % Prefill	nicht getestet, aus
Kürzen alter Tool-Ergebnisse im Agentenkontext	Kontext –39 %, praktisch kein Tempogewinn (Prefix-Cache)	abgelehnt
SGLang (offizielles Cookbook, 1× Spark)	E2E –0.7 %, Eingabegrenze 109–127k mit leerem Stream ohne Fehler , nicht deterministisch bei T=0	verworfen

6.5 Offene Hebel (nur mit Messung)

- Eigenes Draft-Vokabular für nicht-deutsche Arbeitslasten (`draftvocab/README.md`). Für Englisch nicht gemessen.
- `--enable-chunked-prefill` / `--max-num-batched-tokens 8192` , `VLLM_PLE_MMAP_PREWARM` : einzeln nie nachgemessen.
- Neuere Patch-Stände (vLLM 0.30): nur als Herausforderer nach Abschnitt 7.

7 Anderen Runtime- oder Versionsstand testen

Behandle ihn als einmaligen Herausforderer mit schriftlichem Plan: Quellen, exakter Image-Digest, Gates, Abbruchbedingungen, Entscheidungsregel. Nur veröffentlichte, verifizierte Rezepte, keine eigenen Kernel-Ports. Gleiches Kit, gleiche Prompts, ABAB gegen den eingefrorenen Stand. Zusätzlich funktionale Grenzen prüfen: maximale Eingabelänge (inkl. Verhalten bei zu langer Eingabe), Determinismus bei T=0, Tool-Calling. Vorlage: `docs/SGLANG.md` , Rohdaten `data/sglang-challenger-2026-09-25/` .

8 Berichtsformat

Am Ende jeder Aufgabe dem Nutzer berichten, in dieser Struktur:

```

Ergebnis:      <ein Satz, z. B. "GoldVllm läuft, GATES OK" oder "Test abgebrochen, Referenzstand wiederhergestellt">
Zustand jetzt:  Container <name>, Image-ID <id>, Modell-Revision <rev>, Gates <OK/FEHLER>
Geändert:      <Dateien/Einstellungen, mit Sicherungsort>
Gemessen:      <Werte mit Quelle (JSON-Datei)>, getrennt von "abgeleitet" und "laut Upstream"
Nicht gemacht: <übersprungene Schritte und warum>
Risiken/offen: <was der Nutzer entscheiden oder wissen muss>

```

9 Dateikarte

Pfad	Zweck
<code>config/run-goldvllm.sh</code>	einzig gültige Startzeile, prüft Key, Modell, Vokabular-SHA, freie GPU
<code>config/systemd/llm-server.service</code>	Beispiel-Unit (oneshot, RemainAfterExit, wartet auf API) – Platzhalter <code>DEIN_NUTZER</code> ersetzen
<code>config/systemd/llm-server.service.gx10-original</code>	Originalfassung vom Autorgerät, nur zum Vergleich
<code>config/systemd/vllm-tailscale.{service,socket}</code>	optionaler Zugriff über Tailscale, gerätespezifisch

Pfad	Zweck
<code>config/schutz/llm-memory-guard.*</code>	RAM-Wächter (Pflicht)
<code>config/schutz/llm-recovery.{sh,service,timer}</code>	Recovery nach Wächter-Stopp (Abschnitt 4)
<code>config/schutz/llm-recovery.default</code>	Vorlage für <code>/etc/default/llm-recovery</code> (<code>LLMR_KEYFILE</code> Pflicht)
<code>config/serve.sh.goldvllm.patch</code>	Änderung an <code>scripts/serve.sh</code> des Patch-Repos
<code>build/baue_b_inner.sh</code>	Dokumentation, wie das Referenz-Image tatsächlich gebaut wurde
<code>draftvocab/</code>	Vokabular + Rezept (<code>tools/build_draft_vocab.py</code> aus dem Patch-Repo)
<code>bench/</code>	Mess- und Prüfkrit, <code>qualitaet_v1.json</code> ist eingefroren
<code>data/</code>	Rohdaten: Baseline, C_de-Qualifikation, SGLang-Herausforderer, Modell-Manifest
<code>docs/</code> (EN), <code>de/</code> (DE)	ausführliche Doku mit Quellen und Begründungen je Einstellung